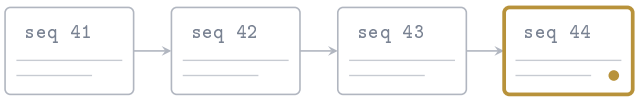
SeBatt: Secure Battery Identity Chip pada FPGA

Riwayat Tegangan Baterai Bertanda Tangan HMAC-SHA-256, Perakitan, dan Penggunaan

Guntur Sulistyo
Muhammad Budi Setiawan
Muhammad Muqtada Alhaddad

Tim NaS, Universitas Gadjah Mada



Pengantar

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat-Nya buku *SeBatt: Secure Battery Identity Chip pada FPGA* ini dapat diselesaikan.

Baterai motor listrik pada jaringan stasiun penukaran berpindah tangan setiap kali ditukar. Riwayat pemakaiannya, misalnya pernah terkuras dalam, menentukan nilai sisa dan klaim garansi, tetapi riwayat tersebut dapat diubah atau di-reset tanpa diketahui stasiun. SeBatt menandatangani setiap sampel tegangan dengan HMAC-SHA-256 beserta nomor urutnya, sehingga stasiun dapat memeriksa apakah riwayat sebuah baterai masih utuh.

Buku ini membahas SeBatt mulai dari konsep dasar dan protokolnya, rancangan modul Verilog, dan perakitan rangkaian pembacaan tegangan, sampai penggunaan stasiun dan hasil pengujian di perangkat keras. Purwarupa yang dibahas berjalan sebagai RTL pada FPGA AMD Kria KV260, membaca baterai LiPo melalui ADC ADS1115, dan diperiksa oleh program stasiun berbasis Python. Kode lengkap semua modul dimuat pada lampiran, sehingga pembaca dapat mengikuti dan mengulangi setiap langkah.

Buku ini ditujukan bagi mahasiswa, peneliti, dan pengembang di bidang sistem digital dan sistem baterai yang ingin menjalankan ulang atau melanjutkan purwarupa SeBatt. Pembaca diharapkan sudah mengenal dasar Verilog dan Python, tetapi tidak harus sudah memahami HMAC maupun protokol stasiun penukaran.

Purwarupa SeBatt dikembangkan dalam Hackathon Deeptech 2026: *Designing Chips for Real-World Impact* yang diselenggarakan oleh ICDeC ITB. Penulis mengucapkan terima kasih kepada penyelenggara, para mentor, serta semua pihak yang telah membantu.

Penulis berharap buku ini bermanfaat bagi pembaca. Kritik dan saran untuk perbaikan buku ini sangat penulis harapkan.

Bandung, 22 Oktober 2026

Tim NaS

Daftar Isi

Pengantar **i**

1 Pendahuluan **1**

2 Konsep Dasar **2**
2.1 Tanda Tangan HMAC-SHA-256 2
2.2 Nonce dan Serangan *Replay* 3
2.3 Nomor Urut Monoton 3
2.4 Pembacaan Tegangan Baterai 4

3 Arsitektur dan Protokol **5**
3.1 Komponen Sistem 5
3.2 Format Frame dan Perintah 7
3.3 Alur Penukaran Baterai 9
3.4 Kode Hasil Pemeriksaan 10

4 Implementasi RTL **12**
4.1 Modul `hmac_sha256` 13
4.2 Modul `auth_ctrl` 14
4.3 Modul `ads1115_ctrl` dan `i2c_master` 17
4.4 Modul `uart_if` dan `kv260_top` 19
4.5 Verifikasi 20

5 Perakitan Perangkat **22**
5.1 Daftar Perangkat dan Bahan 22
5.2 Rangkaian Pembacaan Tegangan 22
5.3 Sambungan ke Header Pmod J2 22
5.4 Perangkat Lunak Stasiun 22

6 Panduan Penggunaan **22**
6.1 Memuat *Bitstream* ke KV260 22
6.2 Pemeriksaan Awal 23
6.3 Menjalankan *Dashboard* 23
6.4 Skenario Penukaran Normal 23
6.5 Skenario Baterai Terkuras Dalam 23
6.6 Skenario Serangan 23
6.7 Penguncian Chip dan Mode Terminal 23
6.8 Kalibrasi Faktor Pembagi 23

7 Hasil Uji di Perangkat Keras **24**
7.1 Susunan Uji 24
7.2 Kinerja Autentikasi dan Pembacaan Riwayat 24
7.3 Deteksi Serangan 24

8 Pemecahan Masalah dan Peringatan **24**
8.1 Pemecahan Masalah 24
8.2 Peringatan Penggunaan 24
8.3 Batasan Purwarupa 24

Daftar Pustaka **25**

A Daftar Istilah **26**

B Daftar Port Modul **26**

C Kode Sumber **26**
C.1 Modul Puncak 26
C.2 Modul Autentikasi dan Komunikasi 26
C.3 Modul Kriptografi 26
C.4 Modul Pembacaan ADC 26
C.5 Constraint KV260 26

1 Pendahuluan

Jumlah kendaraan listrik di Indonesia mencapai 541.678 unit pada Agustus 2026 [1]. Sebagian sepeda motor listrik memakai baterai yang ditukar di stasiun penukaran. SWAP Energi mengoperasikan lebih dari 1.500 stasiun [2], Electrum lebih dari 400 stasiun [3], dan V-Green lebih dari 2.000 titik penukaran [4]. Pertumbuhan ini juga diperhitungkan dalam perencanaan ketenagalistrikan nasional. Rencana Umum Ketenagalistrikan Nasional (RUKN) memproyeksikan kebutuhan listrik naik dari sekitar 539 TWh pada 2025 menjadi sekitar 1.813 TWh pada 2060, dengan kendaraan bermotor listrik menyumbang sekitar 11% dari kebutuhan tahun 2060 [5]. Rencana Usaha Penyediaan Tenaga Listrik (RUPTL) PT PLN (Persero) 2025–2034 memasukkan kendaraan listrik sebagai salah satu pendorong pertumbuhan kebutuhan listrik [6] dan mencakup penyediaan stasiun penukaran baterai kendaraan listrik umum (SPBKLU) untuk kendaraan roda dua [7].

Pada model penukaran, baterai dimiliki operator, sehingga satu baterai berpindah dari satu pengendara ke pengendara lain setiap kali ditukar. Harga satu baterai mencapai Rp4,55 juta [4]. Nilai sisa dan klaim garansinya bergantung pada riwayat pemakaian, misalnya berapa kali baterai pernah terkuras di bawah batas aman.

Gauge baterai yang beredar sudah merekam riwayat pemakaian. Sebagai contoh, BQ40Z50 dari Texas Instruments menyimpan *Lifetime Data* dan *Black Box Recorder*, tetapi riwayat tersebut hanya dilindungi mode akses `SEALED` dan `UNSEALED` serta dapat dihapus melalui perintah *Lifetime Data Reset* [8]. Autentikasi pada *gauge* tersebut hanya membuktikan identitas baterai, sedangkan data ukurnya tidak ikut ditandatangani. Akibatnya, stasiun yang membaca riwayat tidak dapat membedakan riwayat asli dari riwayat yang telah diubah atau di-*reset*.

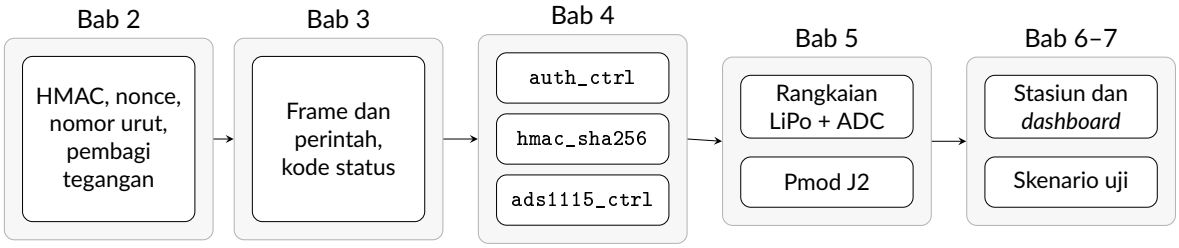
SeBatt menandatangani setiap sampel tegangan di dalam rangkaian yang sama dengan pembacanya. Setiap sampel diberi nomor urut yang hanya bertambah, lalu ditandatangani dengan HMAC-SHA-256 bersama identitas baterai. Saat baterai ditukar, stasiun meminta seluruh riwayat dengan nonce baru dan menghitung ulang setiap tanda tangan, sehingga catatan yang diubah, dihapus, atau diputar balik akan ditolak. Buku ini memandu pembuatan dan penggunaan SeBatt sampai terbukti bekerja di FPGA. Setiap bagian dikerjakan dengan urutan yang sama, yaitu konsep, protokol, modul Verilog, verifikasi dengan model Python dan *testbench* cocotb, lalu uji di board.

Modul yang dibahas adalah `auth_ctrl`, `hmac_sha256`, `sha256_core`, `uart_if`, `ads1115_ctrl`, dan `i2c_master`, yang dirangkai oleh `sebatt_top` dan `kv260_top`. Semua modul ditulis dalam Verilog standar tanpa IP vendor dan tanpa prosesor lunak. Rancangan dijalankan pada board AMD Kria KV260 dengan *clock* 100 MHz dari PS. Tegangan baterai LiPo 3S dibaca melalui modul ADC ADS1115, sedangkan stasiun berjalan sebagai program Python di laptop yang terhubung melalui adaptor UART CP2102. Purwarupa ini masih memakai kunci tetap, penghitung nomor urut yang kembali ke nol saat daya dicabut, dan ADC berupa modul terpisah. Batasan tersebut dibahas pada Bab 8.

Pembaca diasumsikan sudah mengenal dasar Verilog, seperti `module`, `reg`, dan blok `always`, pernah memakai Vivado, serta dapat menjalankan program Python dari terminal. Pembaca tidak harus sudah memahami HMAC, nonce, atau protokol stasiun penukaran, karena konsep tersebut dijelaskan pada Bab 2.

Gambar 1 memperlihatkan hubungan antarbab. Bab 2 menjelaskan tanda tangan HMAC, nonce, nomor urut, dan pembacaan tegangan. Konsep tersebut dipakai Bab 3 untuk menyusun format frame, alur penukaran, dan kode status. Bab 4 mewujudkan protokol tersebut menjadi modul Verilog beserta verifikasinya. Bab 5 merakit rangkaian pembacaan tegangan dan sambungannya ke board. Bab 6 memandu penggunaan stasiun dalam skenario normal

maupun serangan. Hasil pengujiannya di board dimuat pada Bab 7. Pemecahan masalah, peringatan, dan batasan purwarupa dimuat pada Bab 8.



Gambar 1: Peta buku dan bagian yang dibangun pada setiap bab.

2 Konsep Dasar

Bab ini menjelaskan empat konsep yang dipakai di seluruh buku, yaitu tanda tangan HMAC-SHA-256, nonce, nomor urut monoton, dan pembacaan tegangan baterai melalui pembagi tegangan. Setiap konsep langsung dikaitkan dengan perannya di SeBatt dan dengan kode status yang dilaporkan stasiun. Istilah lain yang muncul di buku ini dirangkum pada Lampiran A.

2.1 Tanda Tangan HMAC-SHA-256

Tanda tangan pada SeBatt dibuat dengan HMAC, yaitu fungsi yang menggabungkan kunci rahasia K dengan sebuah pesan m melalui fungsi hash [9]. Fungsi hash yang dipakai adalah SHA-256, yang menghasilkan keluaran 32 byte [10]. HMAC dihitung dengan dua kali pemanggilan hash:

$$\mathrm{HMAC}(K, m) = H\big((K \oplus opad) \,\|\, H((K \oplus ipad) \,\|\, m)\big) \tag{1}$$

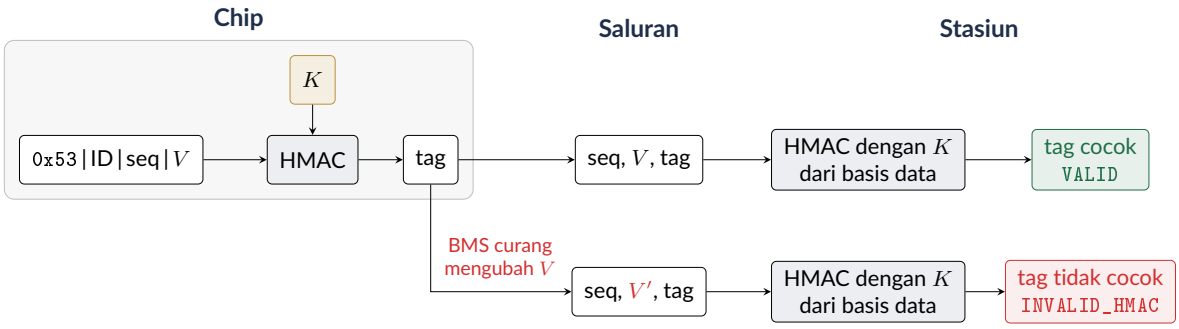
dengan H fungsi SHA-256, $\|$ penyambungan byte, serta $ipad$ dan $opad$ konstanta 64 byte bernilai `0x36` dan `0x5C` yang diulang. Keluaran HMAC disebut *tag*.

Ada dua sifat tag yang dipakai SeBatt. Pertama, perubahan satu bit saja pada pesan menghasilkan tag yang sama sekali berbeda. Kedua, tag yang benar tidak dapat dibuat tanpa mengetahui K. Kunci K sepanjang 32 byte tersimpan di chip dan di basis data stasiun, dan tidak pernah dikirim melalui saluran komunikasi.

Setiap sampel tegangan ditandatangani saat sampel diambil. Pesan yang ditandatangani terdiri atas satu byte prefiks `0x53`, identitas baterai 4 byte, nomor urut 4 byte, dan tegangan 2 byte:

$$tag = \mathrm{HMAC}\big(K,\ \texttt{0x53} \,\|\, ID \,\|\, seq \,\|\, V\big) \tag{2}$$

Stasiun menghitung ulang persamaan (2) untuk setiap catatan yang diterimanya. Apabila sebuah tegangan diubah setelah keluar dari chip, tag hasil hitungan stasiun tidak lagi cocok dengan tag di catatan, sehingga riwayat ditolak dengan status `INVALID_HMAC`. Gambar 2 memperlihatkan kedua kemungkinan tersebut. Prefiks `0x53` membedakan pesan catatan dari dua pesan bertanda tangan lain pada Bab 3, sehingga tag dari satu jenis pesan tidak dapat dipakai sebagai tag jenis pesan lainnya.

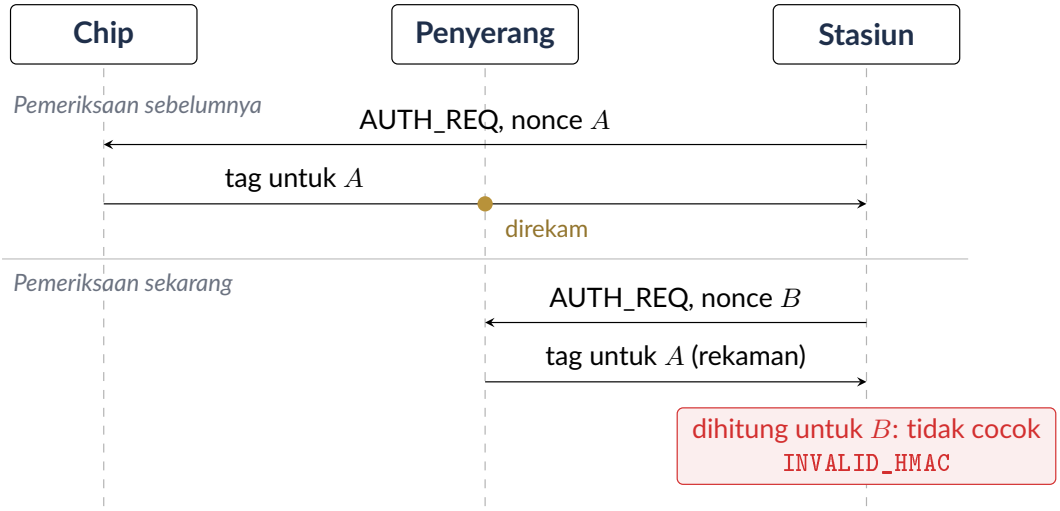


Gambar 2: Penandatanganan dan pemeriksaan tag catatan.

2.2 Nonce dan Serangan *Replay*

Tag pada persamaan (2) membuktikan bahwa sebuah catatan berasal dari chip, tetapi belum membuktikan bahwa catatan tersebut dikirim saat ini. Penyerang dapat merekam balasan chip pada suatu waktu, lalu memutarnya ulang di kemudian hari, misalnya untuk menutupi riwayat yang terjadi sesudah rekaman dibuat. Serangan semacam ini disebut *replay*.

Untuk mencegahnya, stasiun membuat nonce, yaitu bilangan acak 32 byte yang baru untuk setiap pemeriksaan. Nonce dikirim ke chip dan ikut ditandatangani di dalam balasan. Balasan hasil rekaman memuat nonce lama, sehingga tag-nya tidak cocok dengan nonce baru yang sedang diperiksa stasiun dan balasan tersebut ditolak dengan status `INVALID_HMAC`, seperti pada Gambar 3. Stasiun juga mencatat setiap nonce yang sudah dipakai. Pemeriksaan yang memakai nonce yang sama untuk kedua kalinya ditolak dengan status `REPLAY_DETECTED`.



Gambar 3: Serangan *replay*.

2.3 Nomor Urut Monoton

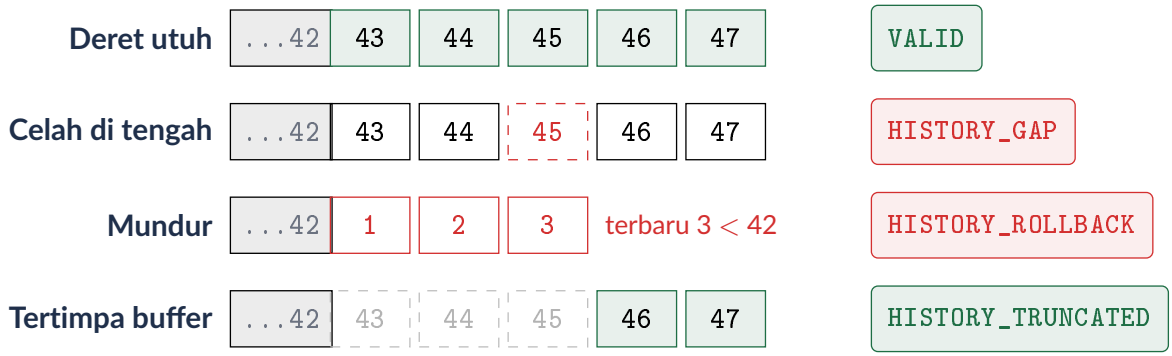
Tanda tangan per catatan belum cukup untuk mendeteksi catatan yang dihapus, karena catatan yang tersisa tetap memiliki tag yang sah. Karena itu, setiap sampel diberi nomor urut (*seq*) dari penghitung 32 bit di chip. Penghitung ini hanya bertambah satu setiap kali sampel diambil dan tidak dapat diatur ulang melalui perintah apa pun. Akibatnya, riwayat yang utuh selalu berupa deret seq yang berurutan tanpa celah.

Saat membaca riwayat, chip juga mengirim tag ringkasan yang mengikat nonce, identitas baterai, seq tertua, dan seq terbaru di dalam buffer. Dengan tag ini stasiun mengetahui rentang seq yang seharusnya diterima. Basis data stasiun menyimpan seq terakhir yang te-

lah diverifikasi untuk setiap baterai. Dari dua informasi tersebut, stasiun membedakan empat keadaan berikut.

- **Deret utuh.** Seq berurutan dari seq tertua sampai seq terbaru dan setiap tag cocok. Riwayat diterima dengan status `VALID`.
- **Celah di tengah.** Ada seq yang hilang atau jumlah catatan tidak sesuai rentang yang ditandatangani. Riwayat ditolak dengan status `HISTORY_GAP`.
- **Mundur.** Seq terbaru lebih kecil daripada seq terakhir di basis data, misalnya karena chip dikembalikan ke keadaan lama. Riwayat ditolak dengan status `HISTORY_ROLLBACK`.
- **Tertimpa buffer.** Buffer chip hanya menampung 64 catatan, sehingga catatan tertua dapat tertimpa sebelum stasiun sempat membacanya. Selama sisa catatan berurutan dan seluruh tag cocok, riwayat tetap diterima dengan status `HISTORY_TRUNCATED`.

Gambar 4 memperlihatkan keempat keadaan tersebut untuk basis data yang terakhir memverifikasi seq 42. Kotak abu-abu adalah riwayat yang sudah diverifikasi, kotak putus-putus merah adalah catatan yang hilang, dan kotak putus-putus abu-abu adalah catatan yang tertimpa sebelum dibaca.



Gambar 4: Empat keadaan riwayat berdasarkan nomor urut.

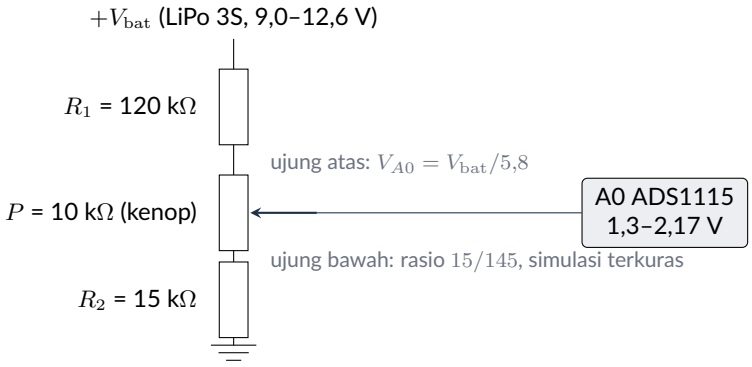
Memotong catatan terbaru juga terdeteksi. Seq terbaru ikut ditandatangani di dalam tag ringkasan, sehingga rentang yang dipersempit membuat tag ringkasan tidak cocok dan riwayat ditolak dengan status `INVALID_HMAC`.

2.4 Pembacaan Tegangan Baterai

Tegangan baterai LiPo 3S berada di antara 9,0 V saat batas terkuras dan 12,6 V saat penuh, jauh di atas batas masukan ADC sebesar 3,3 V. Tegangan tersebut diturunkan lebih dulu dengan pembagi tegangan yang tersusun atas resistor R_1, potensiometer P, dan resistor R_2. Dengan potensiometer di ujung atas, tegangan yang masuk ke ADC adalah

$$V_{A0} = V_{\text{bat}} \cdot \frac{P + R_2}{R_1 + P + R_2} = V_{\text{bat}} \cdot \frac{10 + 15}{120 + 10 + 15} = \frac{V_{\text{bat}}}{5{,}8} \tag{3}$$

sehingga tegangan 9,0–12,6 V menjadi 1,55–2,17 V di masukan ADC. Memutar potensiometer ke bawah memperkecil rasio pembagi sampai $15/145$, seperti pada Gambar 5.



Gambar 5: Pembagi tegangan dengan kenop.

Cara ini dipakai untuk memperagakan baterai yang terkuras tanpa benar-benar menguras sel LiPo, sebagaimana dijelaskan pada Bab 5.

Tegangan pada masukan A0 diubah menjadi bilangan 16 bit oleh ADS1115 [11]. Dengan rentang $\pm$4,096 V, satu LSB setara dengan 0,125 mV, sehingga nilai dalam milivolt diperoleh dengan menggeser hasil mentah tiga bit ke kanan:

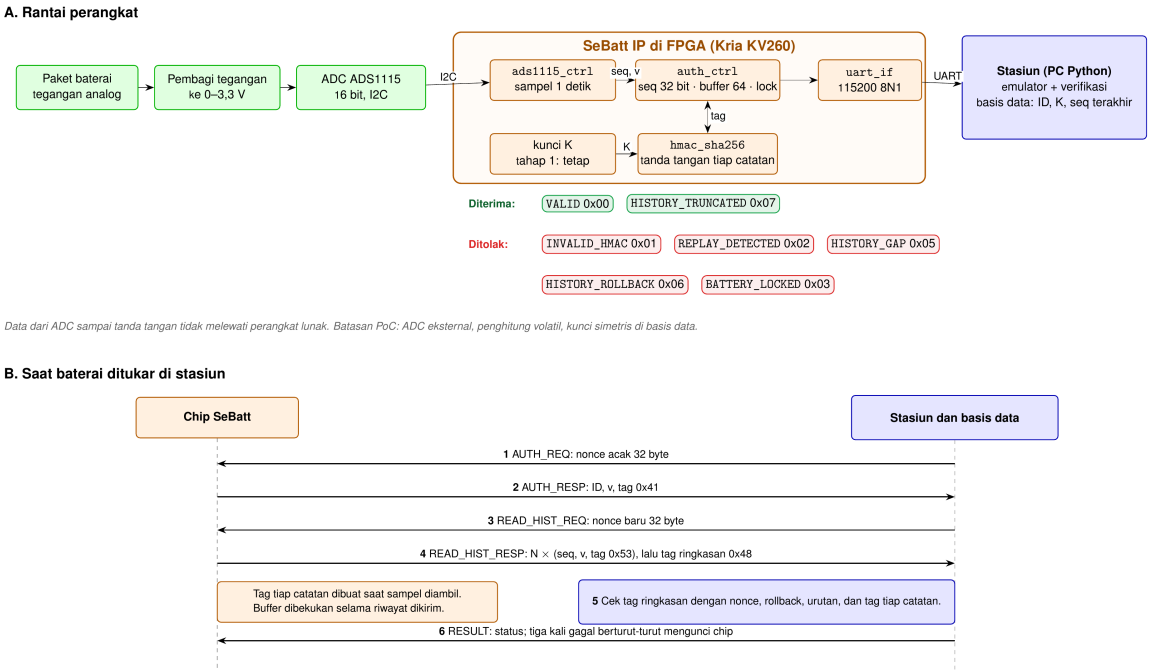
$$V_{A0}\,[\mathrm{mV}] = \frac{raw}{8} \tag{4}$$

FPGA membaca nilai tersebut melalui I2C dan menandatanganinya apa adanya. Nilai yang ditandatangani adalah milivolt di pin A0, bukan tegangan baterai. Stasiun kemudian mengalikannya dengan faktor 5,8 dari persamaan (3) untuk menampilkan tegangan baterai dan menilai kesehatannya. Dengan pembagian tugas ini, faktor tersebut dapat dikalibrasi tanpa mengubah rancangan di FPGA.

3 Arsitektur dan Protokol

3.1 Komponen Sistem

Purwarupa SeBatt terdiri atas tiga bagian, yaitu perangkat keras di sisi baterai, IP SeBatt yang berjalan di FPGA, dan perangkat lunak stasiun di laptop. Gambar 6 memperlihatkan rantai ketiganya. Bagian A menunjukkan jalur data dari baterai sampai stasiun, sedangkan bagian B menunjukkan urutan pesan saat baterai ditukar, yang dibahas pada Subbab 3.3.



Gambar 6: Rantai perangkat dan urutan protokol SeBatt.

Pada produk, IP SeBatt ditempatkan di dalam chip BMS baterai dan berkomunikasi dengan stasiun melalui bus CAN. Purwarupa ini mengganti bus CAN dengan UART dan menjalankan stasiun beserta basis datanya sebagai program Python di laptop. Protokol yang dipakai tetap sama, sehingga penggantian saluran komunikasi tidak mengubah cara pemeriksaan riwayat.

Perangkat keras

Tabel 1 merangkum perangkat keras yang dipakai beserta sambungannya. Semua sinyal antara FPGA dan rangkaian luar melewati header Pmod J2 pada KV260. Nomor pin mengikuti penomoran Pmod 12 pin, dengan pin 5 dan 11 sebagai GND serta pin 6 dan 12 sebagai 3,3 V.

Komponen	Peran	Sambungan
AMD Kria KV260	Menjalankan IP SeBatt di logika terprogram (PL). Prosesor (PS) hanya menyediakan *clock* 100 MHz dan sinyal reset	Pmod J2
LiPo 3S dan pembagi tegangan	Sumber tegangan yang dicatat. Kenop pada pembagi dipakai untuk memperagakan baterai terkuras	Ke A0 ADS1115
Modul ADS1115	ADC 16 bit yang mengubah tegangan A0 menjadi milivolt	I2C, Pmod 1–2
Adaptor CP2102	Saluran UART 115200 8N1 antara FPGA dan laptop	Pmod 7–8
LED hijau dan merah	Hasil RESULT terakhir dari stasiun, merah juga saat chip terkunci	Pmod 9 dan 10
LED *heartbeat*	Berkedip sekitar 6 Hz sebagai tanda rancangan di PL berjalan	Pmod 4
Kabel ke GND	Memilih ADS1115 sebagai sumber sampel	Pmod 3

Tabel 1: Perangkat keras purwarupa SeBatt.

Pin Pmod 3 memilih sumber sampel. Apabila pin ini terhubung ke GND, sampel diambil dari ADS1115. Apabila terhubung ke 3,3 V, chip mencatat nilai tetap 3000 mV, yang berguna

untuk menguji jalur UART tanpa rangkaian ADC. Pin ini tidak boleh dibiarkan terbuka, karena pin yang terbuka terbaca sebagai logika 1. Rangkaian lengkapnya dibahas pada Bab 5.

IP SeBatt di FPGA

IP SeBatt ditulis dalam Verilog standar tanpa IP vendor dan tanpa prosesor lunak. Modul puncak `kv260_top` menerima *clock* dari PS, menyelaraskan reset terhadap *clock*, dan mengeluarkan LED *heartbeat*. Di bawahnya, `sebatt_top` menghubungkan dua bagian utama. Bagian pertama adalah `ads1115_ctrl`, yang membaca ADC melalui `i2c_master`. Bagian kedua adalah `auth_ctrl`, yang mengambil sampel setiap 1 detik, memberi nomor urut, menandatangani sampel dengan `hmac_sha256`, menyimpan 64 catatan terakhir, dan melayani perintah stasiun melalui `uart_if`.

Jalur dari ADC sampai tanda tangan seluruhnya berupa rangkaian logika, tanpa perangkat lunak di antaranya. Identitas baterai pada purwarupa ini bernilai `0x42415401`, yaitu teks `BAT` diikuti byte `0x01`. Rincian setiap modul dibahas pada Bab 4.

Perangkat lunak stasiun

Perangkat lunak stasiun terdiri atas dua program Python. Program utama berada di berkas `model/station.py`. Program ini mengirim perintah ke chip, memeriksa setiap tanda tangan, dan mengirim hasil pemeriksaan kembali ke chip. Basis datanya menyimpan empat hal untuk setiap baterai, yaitu kunci K, seq terakhir yang telah diverifikasi, status kunci, dan nonce yang sudah dipakai. Program ini juga menghitung kesehatan baterai dari riwayat yang lolos pemeriksaan, seperti jumlah kejadian terkuras dalam dan siklus pengisian.

Program kedua adalah *dashboard* web di folder `web/`, yang menampilkan hasil pemeriksaan, grafik riwayat tegangan, dan kartu kesehatan baterai. *Dashboard* memakai aturan pemeriksaan dan penilaian kesehatan yang sama dengan `model/station.py`. Program ini dapat berjalan dalam mode FPGA, yang terhubung ke board, atau mode simulasi, yang memakai model chip di laptop untuk latihan tanpa board. Cara menjalankan kedua program dijelaskan pada Bab 6.

3.2 Format Frame dan Perintah

Stasiun dan chip berkomunikasi melalui UART 115200 8N1 dengan frame biner. Stasiun selalu memulai pertukaran, sedangkan chip hanya menjawab. Stasiun mengirim satu frame lalu menunggu balasannya sebelum mengirim frame berikutnya. Setiap frame tersusun atas empat bagian seperti pada Tabel 2.

Bagian	Panjang	Isi
CMD	1 byte	Kode perintah (Tabel 3)
LEN	2 byte	Panjang PAYLOAD dalam byte, *big-endian*
PAYLOAD	LEN byte	Isi perintah, boleh kosong
CRC	1 byte	CRC-8 atas CMD, LEN, dan PAYLOAD

Tabel 2: Susunan frame.

CRC-8 memakai polinomial `0x31` ($x^8 + x^5 + x^4 + 1$) dengan nilai awal `0x00` dan bit paling signifikan diproses lebih dulu. CRC hanya berfungsi mendeteksi galat transmisi pada kabel. CRC dapat dihitung ulang oleh siapa pun, sehingga keamanan data tetap bergantung pada tanda tangan HMAC di dalam PAYLOAD. Sebagai contoh, permintaan STATUS tanpa PAYLOAD dikirim sebagai empat byte `06 00 00 A5`, dan chip yang tidak terkunci dengan satu kegagalan menjawab `06 00 02 00 01 35`.

Tabel 3 merangkum seluruh perintah, dengan angka dalam kurung menyatakan panjang dalam byte. Kode `0x01`, `0x07`, dan `0x06` meminta balasan dari chip, sedangkan RESULT, LOCK,

dan UNLOCK hanya mengubah keadaan chip tanpa dibalas.

Kode	Perintah	PAYLOAD dari stasiun	Balasan chip
`0x01`	AUTH_REQ	nonce (32)	`0x02` AUTH_RESP: ID (4), V (2), tag (32)
`0x07`	READ_HIST_REQ	nonce (32)	`0x08` READ_HIST_RESP, lihat Tabel 4
`0x06`	STATUS	kosong	`0x06`: status kunci (1), jumlah gagal (1)
`0x03`	RESULT	kode status (1)	tidak ada
`0x04`	LOCK	kosong	tidak ada
`0x05`	UNLOCK	kosong	tidak ada
`0x09`	CHIP_LOCKED	–	Pengganti AUTH_RESP atau READ_HIST_RESP saat chip terkunci: ID (4)

Tabel 3: Perintah dan balasan.

RESULT membawa kode status dari Tabel 6 setelah stasiun selesai memeriksa. Status `VALID` dan `HISTORY_TRUNCATED` mengosongkan penghitung kegagalan di chip, sedangkan status lain menambahkannya satu. Pada kegagalan ketiga berturut-turut chip terkunci dan hanya menjawab AUTH_REQ maupun READ_HIST_REQ dengan CHIP_LOCKED sampai menerima UNLOCK atau dayanya dicabut.

Balasan READ_HIST_RESP memuat seluruh isi buffer beserta tag ringkasan dengan susunan pada Tabel 4. Buffer yang penuh berisi 64 catatan, sehingga PAYLOAD terpanjang adalah $1 + 8 + 64 \times 38 + 32 = 2.473$ byte. Dengan 10 bit per byte pada 115200 bit/s, pengiriman frame sepanjang itu memerlukan sekitar 0,21 detik.

Bagian	Panjang	Isi
N	1 byte	Jumlah catatan, 0–64
seq tertua	4 byte	Nomor urut catatan pertama di buffer
seq terbaru	4 byte	Nomor urut catatan terakhir di buffer
catatan	$N \times 38$ byte	Untuk setiap catatan: seq (4), V (2), tag (32)
tag ringkasan	32 byte	HMAC yang mengikat nonce, ID, seq tertua, dan seq terbaru

Tabel 4: Susunan PAYLOAD READ_HIST_RESP.

Chip membuat tiga jenis pesan bertanda tangan. Prefiks satu byte di awal setiap pesan memisahkan ketiganya, sehingga tag dari satu jenis pesan tidak dapat dipakai sebagai tag jenis lain. Tabel 5 merangkum susunannya.

Pesan	Prefiks	Susunan	Panjang
Autentikasi	`0x41`	prefiks, nonce (32), ID (4), V (2)	39 byte
Catatan	`0x53`	prefiks, ID (4), seq (4), V (2)	11 byte
Ringkasan	`0x48`	prefiks, nonce (32), ID (4), seq tertua (4), seq terbaru (4)	45 byte

Tabel 5: Pesan yang ditandatangani dengan HMAC-SHA-256.

Ketiga pesan lebih pendek dari 56 byte, sehingga setiap pemanggilan hash pada persamaan (1) cukup memproses dua blok SHA-256. Akibatnya, satu tanda tangan selalu memerlukan empat blok SHA-256, berapa pun jenis pesannya. Sifat ini dimanfaatkan pada Bab 4 untuk merancang `hmac_sha256` dengan waktu proses yang tetap.

Chip juga menjaga agar frame yang rusak tidak mengacaukan keadaan. Frame dengan CRC salah dibuang tanpa balasan. Frame dengan LEN lebih dari 64 byte tidak dibaca isinya. Frame

yang terputus di tengah jalan dibuang setelah saluran diam selama 5 milidetik, sehingga frame berikutnya kembali dibaca dari awal. Frame yang datang saat chip masih memproses perintah sebelumnya juga dibuang. Pada setiap kejadian tersebut LED merah menyala, dan stasiun yang tidak menerima balasan menganggapnya sebagai kegagalan komunikasi.

3.3 Alur Penukaran Baterai

SeBatt menjalankan dua kegiatan yang terpisah. Chip mencatat sampel tegangan secara berkala tanpa menunggu stasiun, sedangkan pemeriksaan hanya terjadi saat baterai ditukar. Gambar 6 bagian B memperlihatkan urutan pesan pada pemeriksaan tersebut.

Pencatatan sampel

Pada purwarupa, `auth_ctrl` mengambil satu sampel setiap 1 detik. Selang ini diatur oleh parameter `SAMPLE_INTERVAL_MS`, sehingga pada produk dapat diperpanjang sesuai kebutuhan. Setiap sampel melalui tiga langkah berikut.

1. ADS1115 mengubah tegangan A0 menjadi milivolt, lalu nilainya diteruskan ke `auth_ctrl` melalui I2C.
2. Sampel diberi seq berikutnya, kemudian ditandatangani dengan persamaan (2) saat itu juga.
3. Seq, tegangan, dan tag disimpan di buffer 64 catatan. Apabila buffer penuh, catatan tertua ditimpa, sedangkan penghitung seq terus bertambah.

Karena setiap catatan sudah ditandatangani saat disimpan, chip tidak perlu menandatangani ulang seluruh riwayat ketika diminta. Satu pembacaan riwayat cukup memerlukan satu tanda tangan tambahan, yaitu tag ringkasan.

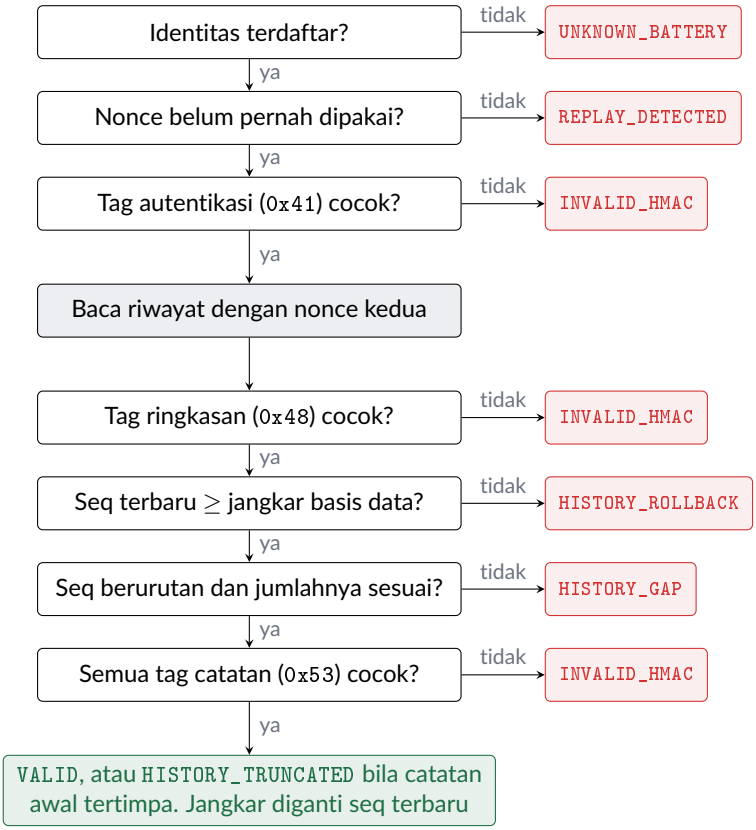
Pemeriksaan saat penukaran

Saat baterai ditukar, stasiun menjalankan lima langkah berurutan.

1. **Autentikasi.** Stasiun membuat nonce baru dan mengirim AUTH_REQ. Chip menjawab dengan identitas baterai, tegangan sampel terakhir, dan tag berprefiks `0x41`. Chip tidak memulai konversi baru untuk menjawab, sehingga balasan selalu memakai sampel yang sudah ada. Stasiun memeriksa bahwa identitas terdaftar, nonce belum pernah dipakai, dan tag cocok. Apabila salah satunya gagal, riwayat tidak dibaca dan pemeriksaan langsung berlanjut ke langkah 4.
2. **Pembacaan riwayat.** Stasiun membuat nonce kedua dan mengirim READ_HIST_REQ. Chip membekukan isi buffer sejak permintaan diterima sampai byte terakhir terkirim. Sampel yang selesai selama pengiriman ditahan dan baru disimpan setelahnya, sehingga satu balasan tidak pernah mencampur catatan lama dan baru.
3. **Verifikasi riwayat.** Stasiun memeriksa balasan dengan urutan pada Gambar 7 dan berhenti pada kegagalan pertama. Identitas yang dipakai adalah identitas yang lolos pada langkah 1.
4. **RESULT.** Stasiun mengirim satu RESULT untuk seluruh penukaran, berisi status akhir pemeriksaan. Chip memperbarui penghitung kegagalan dan LED, lalu terkunci pada kegagalan ketiga berturut-turut.
5. **Jangkar dan kesehatan.** Apabila riwayat diterima, basis data mengganti seq terakhir yang telah diverifikasi dengan seq terbaru, dan kesehatan baterai dihitung hanya dari

catatan yang lolos. Apabila riwayat ditolak, jangkar tidak berubah dan kesehatan baterai tidak ditampilkan.

Chip yang sudah terkunci menjawab AUTH_REQ dengan CHIP_LOCKED, sehingga penukaran berhenti di langkah 1 dengan status `BATTERY_LOCKED`.



Gambar 7: Urutan pemeriksaan di stasiun.

3.4 Kode Hasil Pemeriksaan

Setiap penukaran berakhir dengan satu kode status satu byte. Kode yang sama dikirim ke chip melalui RESULT, dicatat di log stasiun, dan ditampilkan di *dashboard*. Tabel 6 merangkum kedelapan kode beserta penyebab yang umum.

Kode	Status	Hasil	Penyebab umum
0x00	VALID	Diterima	Semua tanda tangan cocok dan seq berurutan sejak pemeriksaan terakhir
0x07	HISTORY_TRUNCATED	Diterima	Catatan awal tertimpa buffer sebelum dibaca, sisa catatan berurutan dan sah
0x01	INVALID_HMAC	Ditolak	Tegangan diubah, rentang riwayat dipotong, balasan lama diputar ulang, atau baterai tiruan tanpa kunci asli
0x02	REPLAY_DETECTED	Ditolak	Nonce yang sama dipakai untuk kedua kalinya
0x03	BATTERY_LOCKED	Ditolak	Chip terkunci setelah tiga kegagalan berturut-turut
0x04	UNKNOWN_BATTERY	Ditolak	Identitas baterai tidak terdaftar di basis data
0x05	HISTORY_GAP	Ditolak	Ada catatan yang dibuang di tengah riwayat
0x06	HISTORY_ROLLBACK	Ditolak	Seq chip lebih kecil dari catatan basis data, misalnya chip direset atau dikembalikan ke keadaan lama

Tabel 6: Kode status yang dilaporkan stasiun.

Hanya dua kode yang dinyatakan diterima. `VALID` berarti seluruh riwayat sejak pemeriksaan terakhir terbukti utuh. `HISTORY_TRUNCATED` juga diterima karena catatan yang hilang tertimpa oleh buffer chip sendiri, bukan dibuang pihak lain. Keadaan ini wajar terjadi pada pembacaan pertama setelah stasiun dijalankan, atau ketika lebih dari 64 sampel diambil di antara dua penukaran. Catatan yang tersisa tetap berurutan dan setiap tanda tangannya cocok, sehingga dapat dipercaya. Pada produk, selang sampel yang lebih panjang membuat 64 catatan mencakup rentang waktu yang jauh lebih lama.

Kedua hasil tersebut berakibat berbeda pada basis data, chip, dan *dashboard*, seperti dirangkum pada Tabel 7.

	Diterima	Ditolak
Basis data	Jangkar diganti seq terbaru	Jangkar tidak berubah
Chip	Penghitung kegagalan kembali ke 0, LED hijau menyala	Penghitung kegagalan bertambah satu, LED merah menyala. Pada kegagalan ke-tiga chip terkunci
Dashboard	Kotak hasil hijau DITERIMA, kartu kesehatan dihitung dari riwayat yang lolos	Kotak hasil merah DITOLAK, kartu kesehatan tidak ditampilkan

Tabel 7: Akibat hasil pemeriksaan.

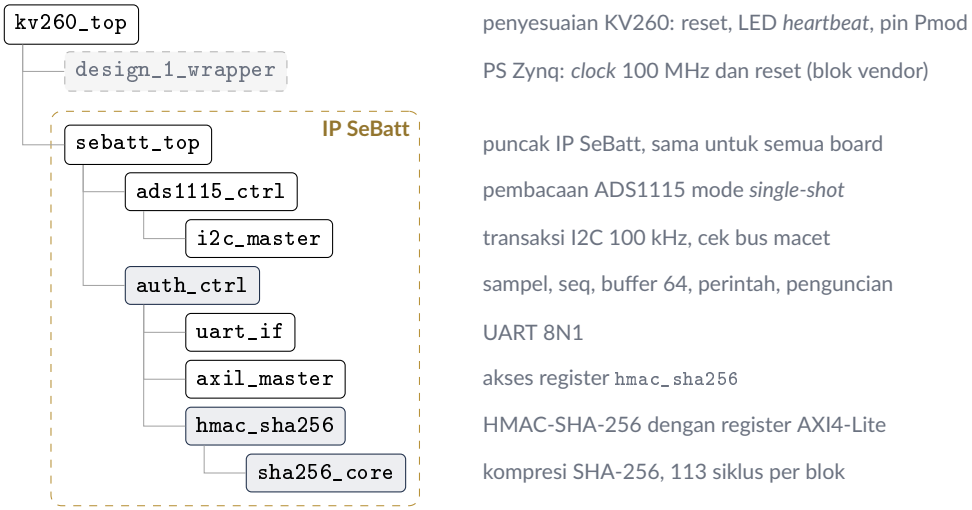
Pengecualiannya adalah `BATTERY_LOCKED`. Chip yang terkunci sudah menjawab dengan CHIP_LOCKED, sehingga stasiun tidak mengirim RESULT dan penghitung kegagalan tidak berubah.

Kode `INVALID_HMAC` mencakup beberapa penyebab yang berbeda. Stasiun tidak perlu membedakannya, karena semuanya berarti data yang diterima tidak dapat dibuktikan berasal dari chip asli. Sebaliknya, `HISTORY_GAP` dan `HISTORY_ROLLBACK` dipisahkan karena tindakannya berbeda. Celah di tengah riwayat menandakan manipulasi, sedangkan seq yang mundur dapat juga berasal dari chip yang dayanya dicabut. Purwarupa ini menyimpan penghitung seq di memori yang hilang saat daya dicabut, sehingga keadaan kedua tidak dapat dibedakan dari manipulasi. Batasan ini dibahas pada Bab 8.

4 Implementasi RTL

Bab ini membahas rancangan IP SeBatt dalam Verilog. Seluruh modul ditulis dengan Verilog standar tanpa IP vendor dan tanpa prosesor lunak. Dengan cara ini, RTL yang sama sudah disintesis untuk KV260 dan Cmod S7 tanpa perubahan pada modul intinya. Semua logika berjalan pada satu domain *clock* dengan reset aktif rendah yang diselaraskan terhadap *clock*. Frekuensi *clock* diberikan sebagai parameter, sehingga modul yang sama dipakai pada KV260 dengan 100 MHz dan pada Cmod S7 dengan 12 MHz.

Gambar 8 memperlihatkan hierarki modul. Bagian di dalam kotak putus-putus adalah IP SeBatt yang sama untuk semua board, dan modul bertepi biru membentuk jalur tanda tangan. Di luarnya, `kv260_top` hanya menyesuaikan IP tersebut dengan KV260. Satu-satunya blok vendor dalam rancangan adalah *block design* PS Zynq, yang dipakai untuk menyediakan *clock* 100 MHz dan sinyal reset. PS tidak menyentuh jalur data, sehingga sampel, tanda tangan, dan perintah seluruhnya diproses di logika terprogram.



Gambar 8: Hierarki modul RTL pada KV260.

Modul `auth_ctrl` menjadi pusat rancangan. Modul ini mengatur pengambilan sampel, penomoran, penyimpanan, penandatanganan, dan pelayanan perintah dari stasiun. Inti kriptografinya berada di `hmac_sha256`, yang memakai antarmuka register AXI4-Lite. Karena itu `auth_ctrl` mengaksesnya melalui `axil_master`, sebuah pengendali AXI4-Lite kecil yang juga ditulis sendiri. Pembacaan ADC terpisah di `ads1115_ctrl`, sehingga sumber sampel dapat diganti tanpa mengubah logika tanda tangan.

Tabel 8 merangkum parameter yang menyesuaikan IP SeBatt dengan board dan kebutuhan demo. Nilai pada kolom KV260 adalah nilai yang dipakai pada purwarupa ini.

Parameter	KV260	Fungsi
`CLK_HZ`	100.000.000	Frekuensi *clock*, dasar pewaktu UART, I2C, dan sampel
`BAUD`	115.200	Kecepatan UART
`BATTERY_ID`	`0x42415401`	Identitas baterai yang ikut ditandatangani
`SAMPLE_INTERVAL_MS`	1.000	Selang antarsampel dalam milidetik
`SAMPLE_SOURCE`	0	Sumber sampel: 0 ADS1115, 1 nilai tetap dari ROM, 2 nilai dari saklar
`RX_TIMEOUT_CYCLES`	`CLK_HZ/200`	Lama saluran diam sebelum frame yang terputus dibuang (5 ms)

Tabel 8: Parameter IP SeBatt.

Selain parameter tersebut, pin `src_sel` memilih sumber sampel saat rancangan berjalan, seperti dijelaskan pada Subbab 3.1. Subbab berikutnya membahas modul satu per satu, dimulai dari inti kriptografi `hmac_sha256`, kemudian `auth_ctrl`, pembacaan ADC, antarmuka UART beserta modul puncak, dan diakhiri dengan verifikasi.

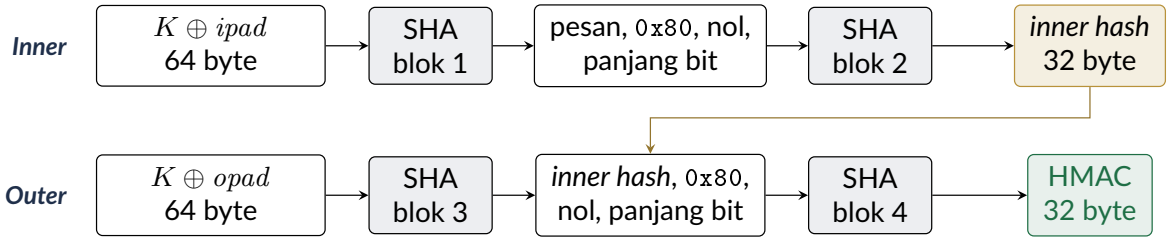
4.1 Modul `hmac_sha256`

Modul `hmac_sha256` menghitung persamaan (1) untuk satu pesan sepanjang paling banyak 55 byte dengan kunci 256 bit. Modul ini dikendalikan melalui register AXI4-Lite pada Tabel 9. Untuk setiap tanda tangan, `auth_ctrl` menulis kunci, pesan, dan panjang pesan, lalu memberi perintah *start*, menunggu bit *done*, dan membaca hasilnya dari register HASH.

Alamat	Nama	Akses	Isi
`0x00`	CTRL	tulis	Bit 0 *start*, bit 1 *init*, keduanya kembali ke 0 dengan sendirinya
`0x04`	STATUS	baca	Bit 0 *done*, bit 1 *busy*
`0x08`	MSG_LEN	baca/tulis	Panjang pesan dalam byte
`0x10–0x2C`	KEY[0..7]	baca/tulis	Kunci 256 bit, 8 kata 32 bit
`0x30–0x6C`	MSG[0..15]	baca/tulis	Pesan, paling banyak 64 byte
`0x70–0x8C`	HASH[0..7]	baca	Hasil HMAC 256 bit

Tabel 9: Peta register `hmac_sha256`.

Satu HMAC dihitung dengan empat blok SHA-256 berurutan seperti pada Gambar 9. Dua blok pertama menghasilkan *inner hash* dari kunci yang di-XOR dengan *ipad* dan pesan. Dua blok berikutnya menghasilkan HMAC dari kunci yang di-XOR dengan *opad* dan *inner hash* tersebut. Mesin keadaan di dalam modul mengisi setiap blok 16 kata ke `sha256_core`, menjalankannya, lalu berpindah ke blok berikutnya tanpa campur tangan `auth_ctrl`.



Gambar 9: Empat blok SHA-256 untuk satu HMAC.

Kunci 32 byte diperpanjang dengan byte nol sampai 64 byte sebelum di-XOR dengan *ipad* (`0x36`) atau *opad* (`0x5C`). Karena delapan kata terakhir kunci bernilai nol, kata tersebut cukup diisi konstanta *ipad* atau *opad*. Potongan kode berikut memperlihatkan pengisian blok pertama.

Kode 1: Pengisian blok $K \oplus ipad$ di `hmac_sha256.v` (baris 149–166).

```
H_LOAD_IPAD: begin
    if (sha_ready) begin
        sha_we   <= 1'b1;
        sha_addr <= lcnt[3:0];
        if (lcnt < 5'd8)
            sha_word <= key_reg[lcnt[2:0]] ^ 32'h36363636;
        else
            sha_word <= 32'h36363636;  // 0x00 ^ 0x36

        if (lcnt == 5'd15) begin
            lcnt        <= 5'd0;
            sha_started <= 1'b0;
            hstate      <= H_RUN_IPAD;
        end else begin
            lcnt <= lcnt + 5'd1;
        end
    end
end
```

Blok kedua memuat pesan diikuti *padding* SHA-256, yaitu byte `0x80`, deretan nol, dan panjang total data dalam bit pada 64 bit terakhir. Panjang total tersebut dihitung sebagai $(64+\textit{panjang pesan})\times 8$ karena blok *ipad* ikut terhitung. Seluruhnya harus muat dalam satu blok 64 byte, sehingga pesan paling panjang 55 byte. Ketiga pesan SeBatt pada Tabel 5 berukuran 11, 39, dan 45 byte, sehingga batas ini selalu terpenuhi. Blok keempat berisi *inner hash* 32 byte dengan panjang total tetap $(64 + 32) \times 8 = 768$ bit.

Perhitungan hash dilakukan oleh `sha256_core`. Modul ini memproses satu blok dalam dua fase. Fase pertama memperluas 16 kata masukan menjadi 64 kata jadwal pesan, satu kata per siklus, sehingga memerlukan 48 siklus. Fase kedua menjalankan 64 ronde kompresi, satu ronde per siklus. Ditambah satu siklus untuk menjumlahkan hasil ke status hash, satu blok selesai dalam 113 siklus. Empat blok untuk satu HMAC memerlukan $4 \times 113 = 452$ siklus kompresi. Ditambah 16 siklus pengisian per blok, satu HMAC memerlukan sekitar 520 siklus, atau sekitar 5,2 μs pada 100 MHz.

Satu ronde per siklus dipilih karena menghemat luas rangkaian. Seluruh 64 ronde memakai satu rangkaian kompresi yang sama secara bergantian, bukan 64 salinan yang dibentangkan. Kecepatan yang lebih tinggi juga tidak diperlukan. Mengirim satu frame AUTH_RESP sepanjang 42 byte melalui UART 115200 bit/s saja memerlukan sekitar 3,6 ms, ratusan kali lebih lama daripada menghitung tanda tangannya.

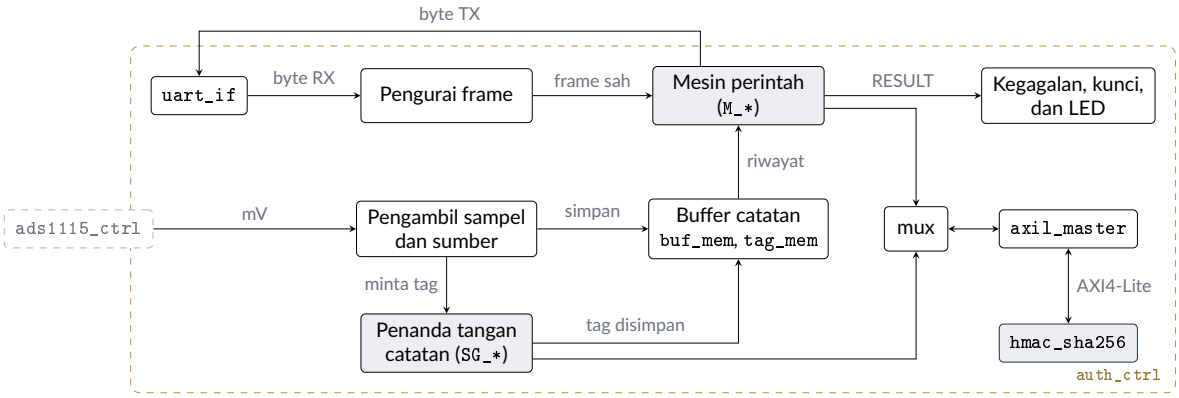
4.2 Modul `auth_ctrl`

Modul `auth_ctrl` menjalankan seluruh perilaku chip yang dijelaskan pada Bab 3. Isinya terbagi menjadi enam bagian yang bekerja bersamaan, seperti pada Gambar 10. Pada gambar tersebut, `ads1115_ctrl` digambar putus-putus karena berada di luar `auth_ctrl`.

1. **Pemilih sumber sampel** memilih ADS1115, nilai tetap dari ROM, atau nilai dari saklar, dan menyelaraskan pin `src_sel` terhadap *clock*.
2. **Pengurai frame** membaca byte dari `uart_if` dan memeriksa CRC sesuai Subbab 3.2.
3. **Pengambil sampel** mengatur selang sampel, memberi seq, dan menyimpan catatan di buffer.
4. **Penanda tangan catatan** menghitung tag setiap catatan yang baru disimpan.
5. **Mesin perintah** melayani AUTH_REQ, READ_HIST_REQ, RESULT, LOCK, UNLOCK, dan

STATUS, lalu mengirim balasannya byte demi byte.

6. **Penghitung kegagalan dan LED** mengikuti RESULT terakhir dan mengunci chip pada kegagalan ketiga.



Gambar 10: Bagian di dalam `auth_ctrl`.

Penyimpanan riwayat

Riwayat disimpan di dua memori yang diindeks oleh posisi yang sama, seperti pada Tabel 10. Penunjuk tulis `buf_wr` berputar dari 0 sampai 63, sehingga catatan ke-65 menimpa posisi catatan pertama. Penghitung `buf_cnt` berhenti di 64, sedangkan `seq_next` terus bertambah. Dari keduanya chip menghitung seq tertua dan seq terbaru yang dikirim pada READ_HIST_RESP.

Register atau memori	Ukuran	Isi
`buf_mem`	64×48 bit	Seq (32 bit) dan tegangan (16 bit) setiap catatan
`tag_mem`	512×32 bit	Tag 256 bit untuk setiap catatan, 8 kata per catatan
`seq_next`	32 bit	Seq untuk sampel berikutnya, hanya bertambah
`buf_wr`	6 bit	Posisi tulis berikutnya, berputar 0–63
`buf_cnt`	7 bit	Jumlah catatan di buffer, berhenti di 64
`fail_cnt`, `locked_r`	5 bit, 1 bit	Penghitung kegagalan dan status kunci

Tabel 10: Penyimpanan di `auth_ctrl`.

Pengambilan dan penandatanganan sampel

Penghitung `sample_cnt` membangkitkan satu *tick* setiap `SAMPLE_INTERVAL_MS`. Pada *tick* tersebut, sumber ADS1115 dimulai melalui `ads1115_ctrl` dan sampel selesai saat ADC menjawab. Sumber ROM dan saklar langsung selesai pada *tick* yang sama tanpa menunggu ADC. Sampel yang selesai segera disimpan ke buffer dan seq bertambah satu. Pembacaan ADC yang gagal tidak menghasilkan catatan, sehingga kegagalan sensor tidak tercatat sebagai tegangan nol.

Setiap catatan yang disimpan menjadi permintaan untuk penanda tangan catatan. Penanda tangan menyusun pesan 11 byte dengan prefiks `0x53`, menulis kunci, pesan, dan panjang pesan ke `hmac_sha256`, menunggu hasilnya, lalu menyimpan delapan kata tag ke `tag_mem` pada posisi yang sama dengan catatannya. Proses ini memerlukan beberapa mikrodetik, jauh lebih singkat daripada selang sampel 1 detik, sehingga permintaan tidak pernah menumpuk.

Berbagi satu `hmac_sha256`

Penanda tangan catatan dan mesin perintah sama-sama membutuhkan HMAC. Keduanya memakai satu `hmac_sha256` yang sama, karena inti SHA-256 merupakan bagian terbesar dari rancangan dan penggunaannya jarang bertumpuk. Pembagiannya diatur oleh multiplexer pada Kode 2. Selama penanda tangan bekerja, jalur ke `axil_master` menjadi miliknya. Mesin perintah baru memakai jalur tersebut setelah tidak ada tag yang sedang atau akan dihitung, dan sinyal `cmd_reserve` mencegah keduanya mengambil jalur pada siklus yang sama.

Kode 2: Multiplexer jalur HMAC di `auth_ctrl.v` (baris 433–436).

```
wire        axi_start = sg_busy ? sg_axi_start : mm_axi_start;
wire        axi_wr    = sg_busy ? sg_axi_wr    : mm_axi_wr;
wire [7:0]  axi_addr  = sg_busy ? sg_axi_addr  : mm_axi_addr;
wire [31:0] axi_wdata = sg_busy ? sg_axi_wdata : mm_axi_wdata;
```

Mesin perintah

Mesin perintah menunggu di keadaan `M_IDLE` sampai pengurai frame menyerahkan frame yang sah. RESULT, LOCK, UNLOCK, dan STATUS diselesaikan langsung di keadaan tersebut. AUTH_REQ dan READ_HIST_REQ memerlukan tanda tangan, sehingga melewati urutan keadaan berikut.

1. `M_WAIT` menunggu sampai sampel terakhir tersedia dan penanda tangan catatan selesai, lalu menyusun pesan. Untuk AUTH_REQ pesannya 39 byte dengan prefiks `0x41`. Untuk READ_HIST_REQ, keadaan ini juga mengambil potret buffer, yaitu jumlah catatan, seq tertua, dan seq terbaru, lalu menyusun pesan ringkasan 45 byte dengan prefiks `0x48`.
2. `M_WR` dan `M_WR_WAIT` menulis kunci, pesan, panjang pesan, dan perintah *start* ke `hmac_sha256`.
3. `M_POLL` dan `M_POLL_WAIT` membaca register STATUS sampai bit *done* bernilai 1.
4. `M_RD` dan `M_RD_WAIT` membaca delapan kata hasil HMAC.
5. `M_TX` dan `M_TX_WAIT` mengirim balasan byte demi byte melalui `uart_if`. CRC dihitung dari byte yang benar-benar terkirim, kemudian dikirim sebagai byte terakhir.

Selama READ_HIST_REQ dilayani, buffer dan `tag_mem` dibekukan sejak potret diambil sampai byte terakhir terkirim. Sampel yang selesai pada rentang tersebut ditahan di satu slot cadangan dan baru disimpan setelah pengiriman selesai. Dengan cara ini isi balasan selalu sesuai dengan rentang seq yang ditandatangani di tag ringkasan.

Penguncian dan LED

RESULT dengan status `VALID` atau `HISTORY_TRUNCATED` mengosongkan `fail_cnt`. Status lain menambahkannya satu, dan ketika nilainya mencapai tiga, `locked_r` menjadi 1. LOCK dan UNLOCK mengubah `locked_r` secara langsung, dan UNLOCK juga mengosongkan `fail_cnt`. Empat keluaran LED diturunkan dari keadaan ini, seperti pada Tabel 11. Pada KV260, `sebatt_top` menyalakan LED hijau di Pmod 9 saat `led[2]` menyala dan `led[3]` padam, serta menyalakan LED merah di Pmod 10 mengikuti `led[3]`.

Keluaran	Menyala ketika
`led[0]`	Berkedip terus sebagai *heartbeat*, sekitar 6 Hz pada 100 MHz
`led[1]`	Mesin perintah, penanda tangan, atau UART sedang bekerja
`led[2]`	RESULT terakhir menyatakan diterima
`led[3]`	Chip terkunci, RESULT terakhir menyatakan ditolak, frame rusak, atau pembacaan ADC terakhir gagal

Tabel 11: Keluaran LED `auth_ctrl`.

4.3 Modul `ads1115_ctrl` dan `i2c_master`

Pembacaan tegangan dibagi menjadi dua modul. Modul `ads1115_ctrl` mengatur langkah pengambilan satu sampel dari ADS1115, sedangkan `i2c_master` menjalankan transaksi I2C yang diminta. Pemisahan ini membuat `i2c_master` tidak bergantung pada jenis ADC, sehingga dapat dipakai ulang untuk perangkat I2C lain.

Langkah pengambilan sampel

Setiap pulsa *start* dari `auth_ctrl` menjalankan empat langkah berikut di `ads1115_ctrl`.

1. Menulis nilai `0xC383` ke register konfigurasi (`0x01`) di alamat I2C `0x48`. Nilai ini sekaligus memulai satu konversi.
2. Menunggu 10 ms. Pada laju 128 sampel per detik, satu konversi memerlukan 7,8 ms.
3. Membaca 16 bit hasil konversi dari register `0x00`.
4. Mengubah hasil mentah menjadi milivolt dengan persamaan (4). Nilai negatif dijadikan nol karena masukan A0 diukur terhadap GND.

Tabel 12 menguraikan arti nilai konfigurasi tersebut [11]. Mode *single-shot* dipilih agar ADC hanya bekerja saat diminta. Dengan cara ini setiap sampel berasal dari konversi yang baru dimulai, dan selang sampel sepenuhnya diatur oleh `auth_ctrl`.

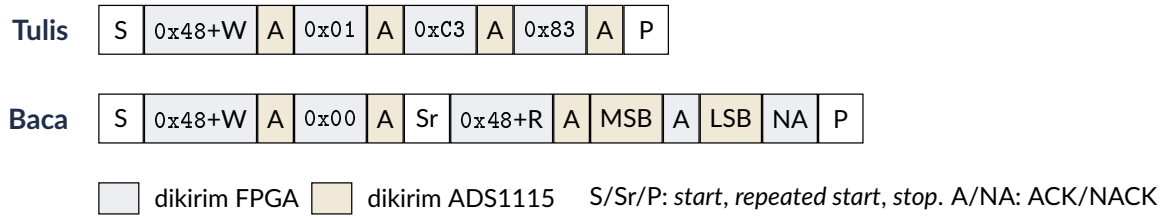
Bit	Medan	Nilai	Arti
15	OS	1	Mulai satu konversi
14–12	MUX	100	AIN0 terhadap GND
11–9	PGA	001	Rentang $\pm$4,096 V, 1 LSB = 0,125 mV
8	MODE	1	*Single-shot*, ADC kembali diam setelah konversi
7–5	DR	100	128 sampel per detik
4–0	COMP	00011	Komparator dimatikan

Tabel 12: Konfigurasi ADS1115 `0xC383`.

Apabila ADS1115 tidak menjawab dengan ACK pada salah satu transaksi, `ads1115_ctrl` menaikkan sinyal *error* dan tidak mengeluarkan sampel. Sinyal ini bertahan sampai pulsa *start* berikutnya. Seperti dijelaskan pada Subbab 4.2, sampel yang gagal tidak disimpan sebagai catatan.

Transaksi I2C

Modul `i2c_master` mendukung dua jenis transaksi pada Gambar 11. Transaksi tulis mengirim alamat register dan dua byte data. Transaksi baca mengirim alamat register, mengulang kondisi *start*, lalu menerima dua byte. Satu sampel memakai satu transaksi tulis dan satu transaksi baca.



Gambar 11: Dua transaksi I2C untuk satu sampel.

SCL dibangkitkan dari *clock* sistem dengan pembagi seperempat bit. Setiap periode SCL dibagi menjadi empat fase, dan perpindahan fase terjadi setiap $\texttt{CLK_HZ}/(4 \times \texttt{I2C_HZ})$ siklus, yaitu 250 siklus pada 100 MHz untuk SCL 100 kHz. Pada kecepatan ini, transaksi tulis memerlukan sekitar 0,4 ms dan transaksi baca sekitar 0,5 ms. Ditambah penantian konversi 10 ms, satu sampel selesai dalam sekitar 11 ms, jauh di bawah selang sampel 1 detik.

Kedua jalur I2C dikendalikan secara *open-drain*. FPGA hanya menarik jalur ke nol atau melepasnya, sedangkan logika satu berasal dari resistor *pull-up* di modul ADS1115. Karena SDA dan SCL datang dari luar chip, keduanya diselaraskan dengan dua *flip-flop* sebelum dibaca. Kode 3 memperlihatkan bagian tersebut.

Kode 3: Jalur *open-drain* dan sinkronisasi di `i2c_master.v` (diringkas dari baris 75–96).

```
assign scl = scl_oen ? 1'b0 : 1'bz;
assign sda = sda_oen ? 1'b0 : 1'bz;

always @(posedge clk) begin
    if (!resetn) begin
        sda_sync <= 2'b11;
        scl_sync <= 2'b11;
    end else begin
        sda_sync <= {sda_sync[0], sda};
        scl_sync <= {scl_sync[0], scl};
    end
end

wire sda_i    = sda_sync[1];
wire bus_idle = sda_sync[1] && scl_sync[1];
```

Sinyal `bus_idle` dipakai untuk memeriksa bus sebelum transaksi dimulai. Transaksi hanya berjalan apabila SDA dan SCL sama-sama tinggi. Apabila salah satunya tertahan rendah, misalnya karena SDA terhubung singkat ke GND, `i2c_master` langsung mengakhiri permintaan dengan tanda gagal. Tanpa pemeriksaan ini, jalur SDA yang tertahan rendah akan terbaca sebagai ACK diikuti data `0x0000`, sehingga sampel 0 mV palsu ikut tersimpan dan ditandatangani seolah merupakan hasil pengukuran.

4.4 Modul `uart_if` dan `kv260_top`

Antarmuka UART

Modul `uart_if` mengirim dan menerima byte dalam format 8N1, yaitu satu bit *start*, delapan bit data dengan bit terendah lebih dulu, tanpa paritas, dan satu bit *stop*. Lebar satu bit adalah `CLK_HZ/BAUD` siklus, dibulatkan ke bawah. Pembulatan ini menghasilkan selisih kecepatan yang jauh di bawah toleransi UART pada kedua board, seperti pada Tabel 13.

Board	Siklus per bit	Kecepatan sebenarnya	Selisih
KV260, 100 MHz	868	115.207 bit/s	+0,006%
Cmod S7, 12 MHz	104	115.385 bit/s	+0,16%

Tabel 13: Pembagi kecepatan UART.

Jalur penerima diselaraskan dengan dua *flip-flop* karena datang dari luar chip. Penerima menunggu tepi turun, lalu memeriksa bit *start* di tengah bit untuk menyaring gangguan singkat. Setiap bit data dibaca di tengah periodenya, sehingga selisih kecepatan kecil antara pengirim dan penerima masih dapat ditoleransi. Byte yang bit *stop*-nya tidak bernilai 1 dibuang tanpa diteruskan ke pengurai frame. Pengirim memakai register geser 10 bit yang berisi bit *start*, data, dan bit *stop*, lalu memberi pulsa *done* setelah bit *stop* selesai dikirim.

Modul puncak `kv260_top`

Modul `kv260_top` menghubungkan IP SeBatt dengan KV260. Fungsinya ada empat.

1. Mengambil *clock* `pl_clk0` 100 MHz dan reset `pl_resetn0` dari *block design* PS (`design_1_wrapper`).
2. Menyelaraskan reset dengan dua *flip-flop*. Reset dari PS tidak sinkron terhadap *clock*, sehingga reset langsung aktif saat diminta tetapi baru dilepas pada tepi *clock*. Dengan cara ini semua register keluar dari reset pada siklus yang sama.
3. Mengeluarkan `led[0]` sebagai LED *heartbeat* di Pmod 4. LED ini berkedip sekitar 6 Hz selama *clock* PL berjalan, sehingga memudahkan memastikan *bitstream* sudah termuat.
4. Memasang parameter `sebatt_top` sesuai Tabel 8 dan meneruskan pin `src_sel` dari Pmod 3.

Purwarupa ini tidak menyediakan tombol reset. Reset mengembalikan seq ke nol, sehingga stasiun akan melaporkan `HISTORY_ROLLBACK` sampai seq kembali melewati jangkar basis data. Untuk memulai ulang, *bitstream* dimuat kembali dan jangkar di stasiun ikut dikosongkan.

Constraint `kv260.xdc`

Berkas `kv260.xdc` memetakan port `kv260_top` ke pin paket FPGA pada header Pmod J2, seperti pada Tabel 14. Semua pin memakai standar I/O LVCMOS33. Berkas ini tidak mendefinisikan *clock*, karena *clock* beserta batasan waktunya sudah ditetapkan oleh *block design* PS.

Pmod	Pin paket	Port	Arah	Catatan
1	H12	`scl`	dua arah	*Pull-up* internal, ditambah *pull-up* modul ADS1115
2	E10	`sda`	dua arah	Sama dengan `scl`
3	D10	`src_sel`	masuk	Disambung ke GND untuk ADS1115
4	C11	`led_hb`	keluar	LED *heartbeat*
7	B10	`uart_tx`	keluar	Ke RXD CP2102
8	E12	`uart_rx`	masuk	*Pull-up* internal agar diam di logika 1 saat CP2102 dilepas
9	D11	`led_green`	keluar	LED hijau
10	B11	`led_red`	keluar	LED merah

Tabel 14: Pemetaan pin Pmod J2.

Nomor Pmod pada tabel mengikuti penomoran Digilent 12 pin, dengan pin 5 dan 11 sebagai GND serta pin 6 dan 12 sebagai 3,3 V. Skematik KV260 memberi label posisi header yang berbeda, sehingga penyambungan sebaiknya selalu mengacu pada penomoran Digilent.

Berkas constraint memasang *pull-down* internal pada `src_sel`. Meskipun demikian, pada uji di board tanggal 21 Oktober 2026, Pmod 3 yang dibiarkan terbuka terbaca sebagai logika 1, sehingga chip memakai sumber nilai tetap 3000 mV. Karena itu Pmod 3 harus selalu disambung ke GND saat memakai ADS1115.

4.5 Verifikasi

Rancangan diverifikasi dalam tiga lapis sebelum dijalankan di board. Lapis pertama adalah model referensi Python yang diuji terhadap vektor resmi. Lapis kedua adalah simulasi RTL dengan *testbench* cocotb, yang membandingkan keluaran rancangan dengan model tersebut. Lapis ketiga adalah sintesis dan implementasi untuk FPGA. Seluruh angka pada subbab ini berasal dari pengujian yang dijalankan ulang pada 22 Oktober 2026.

Model referensi Python

Folder `model/` berisi implementasi SHA-256, HMAC, protokol frame, model chip, dan stasiun dalam Python murni. Model ini menjadi acuan bagi RTL, sehingga diuji lebih dulu terhadap vektor SHA-256 dari FIPS 180-4 [10] dan vektor HMAC-SHA-256 dari RFC 4231 [12]. Simulasi RTL memakai vektor resmi yang sama, dan untuk pesan SeBatt memanggil model ini secara langsung untuk menghitung jawaban yang diharapkan. Tabel 15 merangkum hasil uji model.

Berkas uji	Lolos	Cakupan
`test_model.py`	40/40	Vektor FIPS 180-4 dan RFC 4231, batas pesan 55 dan 56 byte, autentikasi, *replay*, kunci salah, tegangan diubah, penguncian, CRC rusak, model RO-PUF tahap 2
`test_history.py`	19/19	Verifikasi riwayat (utuh, celah, tertimpa, mundur, tag ringkasan diubah), stasiun melalui saluran tiruan, kartu kesehatan untuk tiga profil
`web/test_*.py`	27/27	Logika *dashboard* mode simulasi dan mode FPGA dengan model chip di memori

Tabel 15: Hasil uji model dan stasiun.

Simulasi RTL dengan cocotb

Simulasi dijalankan dengan cocotb 2.1.0 di atas Icarus Verilog 14.0 melalui perintah `make all` di folder `sim/`. Setiap modul diuji terpisah, kemudian `auth_ctrl` diuji bersama seluruh modul di bawahnya melalui *testbench* `tb_auth.v`. Pada uji tersebut, *testbench* berperan sebagai stasiun yang mengirim frame melalui UART, lalu membandingkan setiap balasan dengan hasil model Python. Seluruh 26 uji lolos, seperti pada Tabel 16.

Suite	Lolos	Yang diperiksa
`test_sha256`	5/5	Vektor FIPS, pesan kosong, dua blok, inisialisasi ulang, jumlah siklus per blok
`test_hmac`	4/4	Tiga vektor RFC 4231 dan pesan autentikasi SeBatt
`test_ads1115`	4/4	Pembacaan *single-shot*, nilai negatif dijadikan nol, SDA dan SCL tertahan rendah dilaporkan sebagai galat
`test_auth_ctrl`	13/13	AUTH sesuai model, nonce kedua, CRC salah dan perintah tak dikenal diabaikan, STATUS, CHIP_LOCKED, kunci setelah tiga kegagalan, LED mengikuti RESULT, selang sampel, READ_HIST buffer penuh sesuai model termasuk saat sampel masuk, sumber tanpa ADC, pemulihan pengurai setelah frame rusak

Tabel 16: Hasil simulasi cocotb.

Uji yang paling menentukan adalah READ_HIST dengan buffer penuh saat sampel baru terus masuk. Uji ini memastikan pembekuan buffer pada Subbab 4.2 bekerja, karena tag ringkasan dan isi 64 catatan dari RTL harus sama persis dengan hitungan model untuk rentang seq yang sama.

Sintesis dan implementasi

Rancangan yang sama disintesis dan diimplementasikan untuk Cmod S7 (xc7s25csga225-1) dengan Vivado 2026.1 melalui skrip `fpga/cmod_s7/build.tcl`. Proses selesai tanpa galat dan tanpa *critical warning*, dan pemeriksaan DRC bersih. Tabel 17 merangkum pemakaian sumber daya dan hasil analisis waktunya.

Sumber daya	Terpakai	Tersedia	Persentase
LUT	5.688	14.600	39,0%
sebagai logika	5.276	14.600	36,1%
sebagai memori	412	5.000	8,2%
Register	6.217	29.200	21,3%
Block RAM	0	45	0%
DSP	0	80	0%

Tabel 17: Hasil implementasi pada Cmod S7.

Rancangan tidak memakai Block RAM maupun DSP. Buffer catatan dan memori tag disusun dari LUT sebagai memori terdistribusi, sedangkan seluruh operasi SHA-256 hanya memakai penjumlahan, geser, dan logika bit. Analisis waktu pada *clock* 12 MHz (periode 83,33 ns) menghasilkan WNS +54,826 ns dan WHS +0,027 ns. Kedua nilai positif berarti semua jalur memenuhi batasan waktu, baik untuk *setup* maupun *hold*. Pada KV260, rancangan berjalan dengan *clock* 100 MHz sehingga ruang waktunya lebih sempit. [WNS build KV260 dari laporan timing]

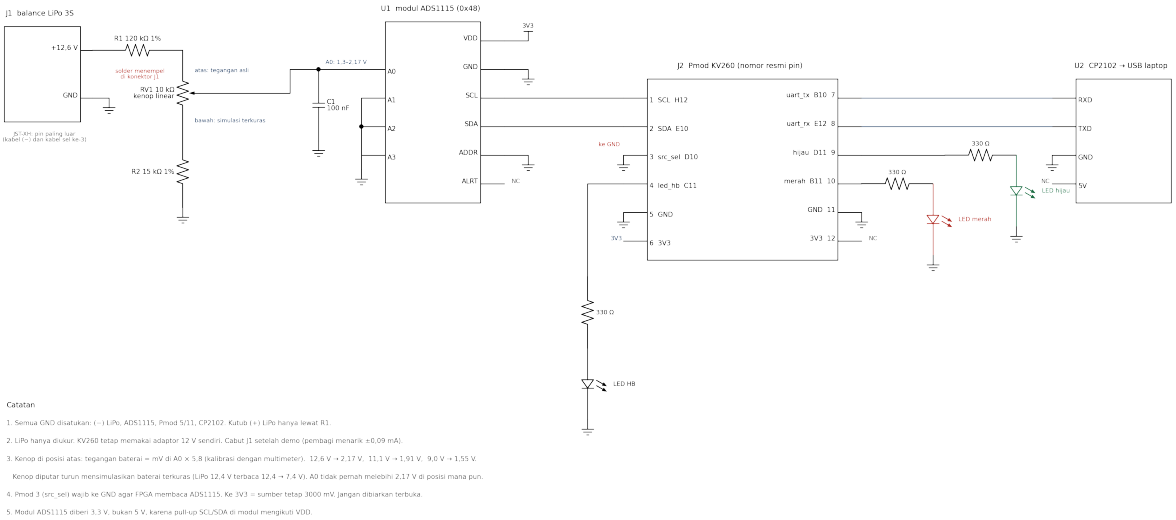
Hasil pengujian di board dibahas pada Bab 7.

5 Perakitan Perangkat

5.1 Daftar Perangkat dan Bahan

[Isi] Tabel: KV260 beserta adaptor 12 V dan kartu SD Ubuntu, kabel micro-USB, ADS1115, LiPo 3S, R1 120 kΩ, kenop 10 kΩ, R2 15 kΩ, kapasitor 100 nF, tiga LED dan resistor 330 Ω, CP2102, kabel jumper, laptop dengan Vivado Hardware Manager dan Python 3.

5.2 Rangkaian Pembacaan Tegangan



Gambar 12: Skematik rangkaian ADC, LED, dan sambungan Pmod J2.

[Isi] Langkah merakit bernomor dan tabel tegangan LiPo terhadap tegangan A0. Ukur wiper sebelum disambung ke A0. Skematik sebaiknya dipindah ke halaman landscape atau dipecah dua.

5.3 Sambungan ke Header Pmod J2

[Isi] Tabel 12 pin beserta nama pin paket. Pmod 3 ke GND, pin 5V CP2102 tidak disambung.

5.4 Perangkat Lunak Stasiun

[Isi] Python 3, pyserial, dan uji tanpa board:

```
py -3.14 -m model.station --offline --demo
```

6 Panduan Penggunaan

6.1 Memuat *Bitstream* ke KV260

[Isi] Boot Ubuntu dari kartu SD, Vivado Hardware Manager, Auto Connect, xck26, Program Device. LED heartbeat berkedip.

6.2 Pemeriksaan Awal

```
py -3.14 -m model.station --check --port COM8
```

[Isi] Contoh keluaran [OK] untuk STATUS, AUTH, READ_HIST.

6.3 Menjalankan *Dashboard*

```
py -3.14 -m web.server --fpga COM8 --profil lipo3s --open
```

[GAMBAR BELUM ADA]
Tangkapan layar dashboard dengan penanda nomor pada tiap bagian

[Isi] Arti tiap bagian dan warna pada dashboard.

6.4 Skenario Penukaran Normal

[Isi] Langkah, hasil DITERIMA, LED hijau, kartu kesehatan terisi.

6.5 Skenario Baterai Terkuras Dalam

[Isi] Kenop diputar turun di bawah 9,0 V lalu kembali. Riwayat tetap DITERIMA dan kartu kesehatan mencatat terkuras dalam.

6.6 Skenario Serangan

[Isi] Hapus catatan (HISTORY_GAP), ubah angka (INVALID_HMAC), muat ulang bitstream (HISTORY_ROLLBACK). Untuk setiap serangan: langkah, hasil, LED, dan tangkapan layar.

6.7 Penguncian Chip dan Mode Terminal

[Isi] Tiga penolakan mengunci chip dan cara membukanya. Perintah interaktif station:

```
py -3.14 -m model.station --port COM8 --interactive --profil lipo3s
```

6.8 Kalibrasi Faktor Pembagi

[Isi] Kenop di ujung atas, ukur LiPo dengan multimeter, faktor = tegangan multimeter dibagi pin mV, isi ke profil lipo3s.

7 Hasil Uji di Perangkat Keras

7.1 Susunan Uji

[GAMBAR BELUM ADA]

Foto susunan uji: KV260, rangkaian LiPo dan ADS1115, CP2102, laptop

7.2 Kinerja Autentikasi dan Pembacaan Riwayat

[Isi] Tabel dari docs/catatan-teknis/hasil-uji-kv260.md: waktu AUTH rata-rata, jumlah autentikasi per detik, waktu READ_HIST 64 catatan, beserta tanggal uji dan bitstream yang dipakai.

Nilai yang belum diukur ditulis sebagai [belum diuji].

7.3 Deteksi Serangan

[Isi] Tabel setiap serangan dan status yang dihasilkan di board.

8 Pemecahan Masalah dan Peringatan

8.1 Pemecahan Masalah

[Isi] Tabel gejala, penyebab, dan tindakan: port COM tidak muncul, tegangan selalu 3000 mV, sampel 0 mV atau berhenti, HISTORY_ROLLBACK setelah memuat ulang bitstream, chip terkunci.

8.2 Peringatan Penggunaan

[Isi] ADS1115 hanya 3,3 V. Kenop tidak boleh langsung ke 12 V. Jangan kuras LiPo di bawah 3,0 V per sel, dan cabut konektor balance setelah dipakai. Pmod 3 tidak dibiarkan terbuka.

8.3 Batasan Purwarupa

[Isi] ADC berupa modul eksternal, penghitung seq volatil, kunci simetris tetap dan tersimpan di basis data, PUF belum terpasang, adegan terkuras dibuat dengan kenop. Pengembangan berikutnya: kunci dari RO-PUF, penghitung di NVM, ADC satu die, sintesis SKY130.

Pustaka

[1] Okezone, "Pengguna Kendaraan Listrik di Indonesia Terus Bertambah," 15 Oktober 2026. https://digital.okezone.com/view/2026/10/15/4/106303/pengguna-kendaraan-listrik-di-indonesia-terus-bertambah

[2] SWAP Energi, situs resmi. https://www.swap.id/

[3] Electrum, "Tentang Kami." https://electrum.id/id/about-us

[4] detikOto, "Begini Sistem dan Biaya Sewa Baterai Motor Listrik VinFast," 2026. https://oto.detik.com/motor-listrik/d-8581711/begini-sistem-dan-biaya-sewa-baterai-motor-listrik-vinfast

[5] Kementerian Energi dan Sumber Daya Mineral, *Keputusan Menteri ESDM Nomor 85.K/TL.01/MEM.L/2025 tentang Rencana Umum Ketenagalistrikan Nasional*, 5 Maret 2025. Ringkasan angka: CNBC Indonesia, "Rencana Umum Ketenagalistrikan Resmi Dirilis, PLTU Batu Bara Masih Ada," 25 Maret 2025. https://www.cnbcindonesia.com/news/20250325131108-4-621534/rencana-umum-ketenagalistrikan-resmi-dirilis-pltu-batu-bara-masih-ada

[6] Kementerian Energi dan Sumber Daya Mineral, *Materi Konferensi Pers RUPTL PT PLN (Persero) Tahun 2025–2034: Meningkatkan Ketahanan dan Keberlanjutan Energi Nasional*, Jakarta, 26 Mei 2025. https://gatrik.esdm.go.id/gatrik/gatrik-api/storage/migrasi/download_index/files/4ec39-materi-paparan-ruptl-2025-2034.pdf

[7] Databoks Katadata, "Projected Number of Electric Vehicles in Indonesia 2025–2034," 3 Juli 2025, merangkum RUPTL PT PLN (Persero) 2025–2034. https://databoks.katadata.co.id/en/transportation-logistics/statistics/68665063883b9/projected-number-of-electric-vehicles-in-indonesia-2025-2034

[8] Texas Instruments, *BQ40Z50-R5 Technical Reference Manual*, SLUUCN4B, bagian 13.3–13.5. https://www.ti.com/lit/ug/sluucn4b/sluucn4b.pdf

[9] H. Krawczyk, M. Bellare, dan R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, IETF, 1997.

[10] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.

[11] Texas Instruments, *ADS111x Ultra-Small, Low-Power, I2C-Compatible, 860-SPS, 16-Bit ADCs*, lembar data SBAS444.

[12] M. Nystrom, *Identifiers and Test Vectors for HMAC-SHA-224, HMAC-SHA-256, HMAC-SHA-384, and HMAC-SHA-512*, RFC 4231, IETF, 2005.

[13] AMD, *Kria KV260 Vision AI Starter Kit User Guide*, UG1089.

A Daftar Istilah

Istilah	Arti
ADC	*Analog-to-Digital Converter*, pengubah tegangan menjadi angka
Bitstream	Berkas konfigurasi yang dimuat ke FPGA
BMS	*Battery Management System*
FPGA	[isi]
HMAC	[isi]
I2C	[isi]
KV260	[isi]
LiPo 3S	[isi]
Nonce	[isi]
Pmod	[isi]
PUF	[isi]
RTL	[isi]
Seq	[isi]
Stasiun penukaran (BSS)	[isi]
UART	[isi]

Tabel 18: Daftar istilah.

B Daftar Port Modul

[Isi] Tabel port untuk kv260_top, sebatt_top, auth_ctrl, hmac_sha256, ads1115_ctrl, i2c_master, dan uart_if: nama, arah, lebar, dan fungsi.

C Kode Sumber

[Isi] Kode lengkap dimuat dengan `\lstinputlisting` dari folder rtl/ dan fpga/kv260/ agar selalu sama dengan repositori. Urutan: kv260_top.v, sebatt_top.v, auth_ctrl.v, uart_if.v, axil_master.v, hmac_sha256.v, sha256_core.v, ads1115_ctrl.v, i2c_master.v, kv260.xdc. Modul key_vault.v dan ro_puf.v untuk tahap 2 tidak termasuk.

C.1 Modul Puncak

C.2 Modul Autentikasi dan Komunikasi

C.3 Modul Kriptografi

C.4 Modul Pembacaan ADC

C.5 Constraint KV260